

VALUE ADDED PROBLEM-2 (Experiment – 12)

/* Name-Manaswi Butle

/PRN-25070521224/

SECTION-D-II

Value Added Problem-2 */

#include <stdio.h>

#include <string.h>

#define MAX 1000000

char stack[MAX];

int top = -1;

// Push function

void push(char c) {

 stack[++top] = c;

}

// Pop function

char pop() {

 if (top == -1)

 return '\0';

 return stack[top--];

}

// Check if matching pair

int isMatchingPair(char open, char close) {

 if (open == '(' && close == ')') return 1;

 if (open == '{' && close == '}') return 1;

```

    if (open == '[' && close == ']') return 1;
    return 0;
}

int isBalanced(char *s) {
    for (int i = 0; s[i] != '\0'; i++) {

        // If opening bracket, push
        if (s[i] == '(' || s[i] == '{' || s[i] == '[') {
            push(s[i]);
        }

        // If closing bracket
        else if (s[i] == ')' || s[i] == '}' || s[i] == ']') {

            if (top == -1) // No matching opening
                return 0;

            char topElement = pop();

            if (!isMatchingPair(topElement, s[i]))
                return 0;
        }
    }

    // If stack is empty → balanced
    return (top == -1);
}

```

```

int main() {

    char s[MAX];


    printf("Enter expression: ");

    scanf("%s", s);


    if (isBalanced(s))

        printf("true\n");

    else

        printf("false\n");


    return 0;

}

```


Online Compiler

Programiz PRO

main.c

Share

Run

Output

Clear

```

1  * Name-Simrat Singh Arora
2  /PRN-25070521181/
3  SECTION-D-I
4  Value Added Problem-2 */
5  #include <stdio.h>
6  #include <string.h>
7
8  #define MAX 1000000
9
10 char stack[MAX];
11 int top = -1;
12
13 // Push function
14 void push(char c) {
15     stack[++top] = c;
16 }
17
18 // Pop function
19 char pop() {
20     if (top == -1)
21         return '\0';
22     return stack[top--];
23 }
24
25 // Check if matching pair
26 int isMatchingPair(char open, char close) {
27     if (open == '(' && close == ')') return 1;
28     if (open == '{' && close == '}') return 1;
29     if (open == '[' && close == ']') return 1;

```

```

Enter expression: {[()]}
true

=== Code Execution Successful ===

```